OptimaTech LMS — Backend API Documentation

Complete request/response reference for every endpoint in the NestJS backend (`backend/src/**/*.controller.ts`). This is the detailed companion to `API_REFERENCE.md` (which is the quick-lookup table). If a route isn't here, it doesn't exist in the API.

> **Mapping the API to the code.** Appendix A lists all 95 routes with a **Source** column linking each to its exact handler (`file:line`). Appendix B maps every controller to its base path and the service that holds its logic. DTOs (request shapes + validation) live in each module's `dto/` folder next to its controller.

---

0. Conventions

Thing	Value
Global path prefix	`/api/v1` (set in `src/main.ts`)
Dev base URL	`http://localhost:3000/api/v1` (backend listens on `PORT ?? 3000`)
Staging	`https://api-staging.optimatech.io/api/v1`
Production	`https://api.optimatech.io/api/v1`
Auth header	`Authorization: Bearer <Firebase ID token>`
Request content type	`application/json` (except mission-template upload — `multipart/form-data`)
Validation	Global `ValidationPipe` with `whitelist` + `forbidNonWhitelisted` — **unknown body properties are rejected with 400**
Rate limit	**100 requests / 60s** per client (`@nestjs/throttler`) → `429` when exceeded. The two `/health*` routes (§1) are exempt.
Data source	Backend → Firebase Data Connect (Postgres). Clients never touch Firestore/Data Connect directly.

Auth levels used below:

- **Public** — no token required.
- **User** — any signed-in user (Firebase token). Every signed-in user is implicitly a `student`.
- **Instructor / Admin** — role-gated via Firebase custom claims. A plain student gets `403`. **Owner** means the instructor must own the course/mission the resource belongs to.

ID format: UUIDs. The API accepts both dashed (`550e8400-e29b-41d4-a716-446655440000`) and compact 32-char hex, **except** `reorder` endpoints whose `orderedIds` **must be dashed v4 UUIDs**.

Standard error shape (NestJS):

```
{ "statusCode": 400, "message": ["rating must not be greater than 5"], "error": "Bad Request" }
```

Common statuses: `400` validation · `401` missing/expired token · `403` wrong role / not owner / not enrolled · `404` not found · `409` conflict · `410` quiz session expired · `429` rate-limited · `502/504` upstream DB.

Sign-up is not REST. Email/password registration and admin role changes go through **Firebase Cloud Functions** (called via the Firebase SDK), not this API — see **§23**. This REST API assumes the caller is already authenticated.

First call after sign-in: `POST /me/sync` once, to provision the user row before any other authenticated call.

> **How to read the response examples.** Fields you consume on the frontend (ratings summary, progress %, quiz grade, upload URLs, streaming payloads, computed/list objects) are shown **exactly** as the API returns them. For plain create/update/delete calls that pass straight through to the database, the response is a thin wrapper like `{ "course_insert": { "id": "…" } }` — treat the `id` as the useful part and don't depend on the wrapper key name for business logic. Every success status is included (`200`, `201`, or `202`). Where an endpoint can return `null` (e.g. "my rating", "my enrollment"), that is called out.

0.5. SDK vs. REST API Routing Guidelines

Our application uses a hybrid architecture combining Firebase Data Connect (FDC) client SDKs with this NestJS backend. To optimize performance and reduce server load, client applications (Flutter, Web) MUST follow these routing rules:

1. Use the FDC Client SDK natively (Skip NestJS)

For all **Public & Student Data-Reading Operations**, use the generated FDC SDK (`mobile/lib/dataconnect_generated`). FDC securely processes these queries with zero middleman latency via `@auth(level: PUBLIC)` or `@auth(level: USER)`.

- **Discovery:** Fetching popular courses, recommended courses, course catalogs.
- **Curriculum:** Loading course details, sections, and lecture metadata.
- **Progress:** Fetching user enrollments and completed lectures.

2. Use this NestJS REST API

For all **Authoring, Heavy Compute, and Complex RBAC**, use the endpoints documented in this file. The backend connects to Postgres using a privileged Admin SDK to bypass `NO_ACCESS` limits on mutations.

- **Instructor Authoring:** Creating courses, uploading videos, editing quizzes (requires deep relational ownership checks).
- **Multimedia:** Video transcoding triggers, HLS streaming key retrieval, and chunked file uploads.
- **High-Frequency Writes:** Video progress playback heartbeats.

---

1. Health

Two probes with different jobs. Both are **Public** and both are **exempt from the global rate limit** — they are polled on a fixed cadence by machines, and a monitor must never be throttled into looking like an outage.

`GET /health`

Liveness probe — "is the process alive?" **Public.**

Response `200`:

```
{
  "status": "ok",
  "timestamp": "2026-07-04T10:15:00.000Z",
  "uptime": 3812,
  "env": "production"
}
```

`uptime` is process uptime in seconds (it resets on restart); `env` is `NODE_ENV`, or `"unknown"` when unset. Deliberately dependency-free — it reports process liveness only, so a transient DB/Redis blip cannot flap Railway's healthcheck. It is **not** a readiness probe; use the one below for that.

This is the endpoint wired to Railway's `healthcheckPath` and to the external uptime monitor. It never returns a non-200 while the process is running.

`GET /health/ready`

Readiness probe — "can this instance serve traffic?" **Public.**

Checks the dependencies a real request needs: **Redis** (the rate limiter fails closed without it, so every route would 503) and **Data Connect** (every read and write goes through it).

Response `200` — ready:

```
{
  "status": "ok",
  "timestamp": "2026-07-28T01:00:00.000Z",
  "checks": {
    "redis":       { "status": "up", "latencyMs": 2 },
    "dataconnect": { "status": "up", "latencyMs": 88 }
  }
}
```

Response `503` — a dependency is unreachable:

```
{
  "status": "error",
  "timestamp": "2026-07-28T01:00:00.000Z",
  "checks": {
    "redis":       { "status": "down", "latencyMs": 3000, "error": "timeout" },
    "dataconnect": { "status": "up", "latencyMs": 91 }
  }
}
```

Response `503` — the process is shutting down (returned before any check runs, so a draining instance leaves the load balancer's rotation immediately):

```
{ "status": "shutting_down", "timestamp": "2026-07-28T01:00:00.000Z" }
```

Field	Meaning
`status`	`ok` · `error` · `shutting_down`
`checks.<dep>.status`	`up` or `down`
`checks.<dep>.latencyMs`	How long that check took, recorded either way
`checks.<dep>.error`	Only when `down`: `timeout` or `unreachable`

`error` is coarse on purpose. This endpoint is unauthenticated and raw driver messages carry internal hosts and ports (`connect ECONNREFUSED 10.x.x.x:6379`); the full reason is written to the server log instead.

Client note: this is an operational endpoint for load balancers, deploy verification and monitoring — **not** something the mobile or web app should call. Results are cached for 10s and each check is timeout-capped at 3s, so polling it faster than that returns a cached

answer.

> Deliberately **not** wired to Railway's `healthcheckPath` — a transient Redis blip would otherwise fail the platform healthcheck and roll back a healthy deploy. See `docs/OPERATIONS.md` §2.1.

---

2. Identity — `me`

All routes require **User** auth. `uid` is always taken from the verified token, never the body.

`POST /me/sync`

Provision/refresh the caller's user row. Call once after first sign-in.

Request body:

```
{
  "email": "jane@example.com",
  "name": "Jane Doe",
  "avatar": "https://cdn.optimatech.io/avatars/jane.png"
}
```

Field	Type	Rules
`email`	string	valid email, required
`name`	string	1–200 chars, required
`avatar`	string	optional, ≤1000 chars

Response `201`: the upserted user record.

```
{ "user_upsert": { "uid": "firebase-uid-123" } }
```

`GET /me`

Current user + profile + roles.

Response `200`:

```
{
  "uid": "firebase-uid-123",
  "email": "jane@example.com",
  "name": "Jane Doe",
  "avatar": "https://cdn.optimatech.io/avatars/jane.png",
  "profile": { "bio": "…", "company": "…", "phone": "…", "preferences": {} },
  "roles": ["student"]
}
```

Returns `null` if the row was never synced.

`PATCH /me/profile`

Update own profile. All fields optional.

Request body:

```
{ "bio": "Backend engineer", "company": "OptimaTech", "phone": "+201234567890", "preferences": { "theme": "dark" } }
```

Field	Type	Rules
`bio`	string	≤2000
`company`	string	≤200
`phone`	string	≤50
`preferences`	object	free-form JSON

Response `200`: `{ "userProfile_upsert": { "id": "firebase-uid-123" } }`

---

3. Administration — `admin/users`

`PUT /admin/users/:uid/role`

Assign a global role by writing the user's `roles` custom claim (the single source of truth for authorization; any legacy `role` / `admin` claims are purged). Takes effect on the user's next token refresh. **Admin only.**

Request body:

```
{ "role": "instructor" }
```

`role` ∈ `admin | instructor | student`.

Response `200`:

```
{ "uid": "firebase-uid-123", "role": "instructor" }
```

Errors: `403` if caller isn't admin · `400` if role invalid.

---

4. Course discovery — `catalog`

`GET /catalog/courses`

Published course cards. **Public.**

Response `200`:

```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "title": "Intro to Cloud",
    "description": "…",
    "price": 4900,
    "language": "en",
    "level": "BEGINNER",
    "thumbnail": "https://…",
    "avgRating": 4.6,
    "ratingCount": 128
  }
]
```

`GET /catalog/courses/:courseId/curriculum`

Course outline (sections → lectures) + rating summary. **User.**

Response `200`:

```
{
  "id": "550e8400-…",
  "title": "Intro to Cloud",
  "sections": [
    {
      "id": "…",
      "title": "Getting Started",
      "order": 1,
      "lectures": [
        { "id": "…", "title": "Welcome", "type": "VIDEO", "duration": 300, "order": 1, "isFreePreview": true }
      ]
    }
  ],
  "ratingSummary": { "average": 4.6, "count": 128, "distribution": { "1": 2, "2": 3, "3": 10, "4": 40, "5": 73 } }
}
```

Errors: `404` if course not found.

---

5. Courses — `courses`

Six **User** discovery / student routes (search, categories, popular, recommended, continue-learning, videos) sit alongside the authoring set. All *authoring* routes are **Instructor/Admin**; mutating a specific course requires **owner** (CourseAccessGuard).

> These discovery routes live on the `courses` controller behind `FirebaseAuthGuard` (no role required), so they need a signed-in token even though most of the underlying data is public catalog content. `popular`, `recommended` and `continue-learning` are declared **before** `:courseId` so the static paths aren't shadowed by the param route.

`GET /courses/search?q=<text>`

Search **published** courses by title (substring match). **User. Query:** `q` — required; `400 Bad Request` if missing or empty. **Response `200`:** array of course cards (empty array if none match):

```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "title": "Intro to Cloud",
```

```
    "price": 4900,
    "level": "BEGINNER",
    "thumbnail": "https://…",
    "tags": ["k8s", "devops"],
    "category": { "name": "Cloud" },
    "author": { "name": "Jane Doe", "avatar": "https://…" }
  }
]
```

`GET /courses/categories`

All platform categories, alphabetical (for dropdowns / filtering). **User. Response** `200` :

```
[ { "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6", "name": "Cloud", "description": "Cloud computing courses" } ]
```

`GET /courses/popular`

Most popular **published** courses, for a discovery carousel. **User.** Popularity is ranked by the denormalised rating aggregate — `ratingCount` first, then `avgRating` , then newest — so the list is a single indexed read with no per-course fan-out. **Query:** `limit` — optional; **default 10, clamped to 1–50** (missing/invalid falls back to 10). **Response** `200` : array of course cards (same shape as `/catalog/courses` , empty array if none):

```
[
  {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "title": "Intro to Cloud",
    "price": 4900,
    "level": "BEGINNER",
    "thumbnail": "https://…",
    "tags": ["k8s", "devops"],
    "avgRating": 4.6,
    "ratingCount": 128,
    "category": { "name": "Cloud" },
    "author": { "name": "Jane Doe", "avatar": "https://…" }
  }
]
```

`GET /courses/recommended`

Personalized course recommendations for the caller. **User.** The backend looks at the categories of the courses the caller is enrolled in (`ACTIVE` / `COMPLETED` enrollments) and returns other **published** courses in those categories that they are **not** already enrolled in, best-rated first. A caller with no enrollment history — or whose categories yield nothing new — transparently **falls back to the popular list**, so the rail is never empty. **Query:** `limit` — optional; **default 10, clamped to 1–50**. **Response** `200` : array of course cards, identical shape to `/courses/popular` .

`GET /courses/continue-learning`

The caller's enrolled courses (`ACTIVE` / `COMPLETED`), each annotated with progress and **sorted by completed-lesson count, highest first** (ties broken by percentage). **User.** Soft-deleted courses are omitted. **Response** `200` :

```
[
  {
    "enrollmentId": "enrollment-uuid",
    "status": "ACTIVE",
    "enrolledAt": "2026-06-01T10:00:00.000Z",
    "completedAt": null,
```

```
    "course": {
      "id": "550e8400-…",
      "title": "Intro to Cloud",
      "thumbnail": "https://…",
      "level": "BEGINNER",
      "price": 4900,
      "category": { "name": "Cloud" },
      "author": { "name": "Jane Doe", "avatar": "https://…" }
    },
    "totalLessons": 20,
    "completedLessons": 12,
    "percentage": 60
  }
]
```

Returns an empty array if the caller isn't enrolled in anything.

`GET /courses/:courseId/videos`

Descriptions of every **uploaded video lecture** in a course (`type: VIDEO`), grouped-flat across sections in curriculum order. **User**, access-gated. The `description` is each video lecture's descriptive copy. Access mirrors the curriculum: the course **owner/admin or an active/completed enrollee** sees every description; any other caller sees the video list but gets `description: null` for non-free-preview videos (paid copy is withheld, free-preview descriptions are always shown). **Response** `200`:

```
{
  "courseId": "550e8400-…",
  "title": "Intro to Cloud",
  "videos": [
    {
      "id": "lecture-uuid",
      "title": "Welcome",
      "description": "A quick tour of what this course covers.",
      "duration": 120,
      "isFreePreview": true,
      "order": 1,
      "sectionId": "section-uuid",
      "sectionTitle": "Getting Started"
    },
    {
      "id": "lecture-uuid-2",
      "title": "Deep Dive",
      "description": null,
      "duration": 600,
      "isFreePreview": false,
      "order": 2,
      "sectionId": "section-uuid",
      "sectionTitle": "Getting Started"
    }
  ]
}
```

Errors: `404` if the course is not found or soft-deleted.

`POST /courses`

Create a course (DRAFT).

Request body:

```
{
  "title": "Advanced Kubernetes",
```

```
  "description": "Deep dive",
  "price": 9900,
  "language": "en",
  "level": "ADVANCED",
  "categoryId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "tags": ["k8s", "devops"]
}
```

Field	Type	Rules
`title`	string	required
`description`	string	optional
`price`	int	≥0 (minor units / cents)
`language`	string	required
`level`	enum	`BEGINNER | INTERMEDIATE | ADVANCED`
`categoryId`	UUID	required
`tags`	string[]	required (may be empty)

Response `201` : `{ "course_insert": { "id": "…" } }`

`GET /courses`

Courses the caller authored. **Response** `200` : array of course objects.

`GET /courses/:courseId`

Full admin view (includes sections/lectures). **Owner. Response** `200` : the full course object. `404` if not found / not owned.

`GET /courses/:courseId/overview`

Instructor analytics for a single course — course details, content counts, enrollment breakdown, completion/progress stats and ratings. **Owner** (admins pass unconditionally). Aggregated server-side from a single privileged read, so it is cheap for the client to render a dashboard.

- **Progress stats are scoped to "learners"** = enrollments with status `ACTIVE` or `COMPLETED`, deduped by user (a re-enrolled student counts once). A lecture counts as done only when its progress row is `completedAt` -stamped.
- `averageCompletionPercent` = mean, over learners, of `completedLectures / totalLectures` (0–100, 2-dp).
- `completionRatePercent` = `fullyCompletedLearners / activeLearners` (0–100, 2-dp), where a *fully-completed* learner has finished every lecture.
- `enrollments.total` / `byStatus` count **raw** enrollment rows (re-enrollments included).
- `content.totalQuizzes` = lectures of type `QUIZ` ; `assignments` = the course's missions plus their submission breakdown.
- `ratings.average` / `count` come from the denormalised course aggregate; `distribution` is the per-star (1–5) histogram.

Response `200` :

```
{
  "course": {
    "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "title": "Advanced Kubernetes",
    "description": "Deep dive",
    "price": 9900,
```

```
    "status": "PUBLISHED",
    "language": "en",
    "level": "ADVANCED",
    "thumbnail": "courses/…/thumb.png",
    "tags": ["k8s", "devops"],
    "category": { "id": "…", "name": "Cloud" },
    "author": { "id": "uid-…", "name": "Sam", "email": "sam@x.io", "avatar": null },
    "createdAt": "2026-01-01T00:00:00Z",
    "updatedAt": "2026-02-01T00:00:00Z"
  },
  "instructors": [
    { "id": "uid-…", "name": "Sam", "email": "sam@x.io", "avatar": null }
  ],
  "content": {
    "totalSections": 4,
    "totalLectures": 22,
    "lecturesByType": { "VIDEO": 16, "READING": 4, "INTERACTIVE": 0, "QUIZ": 2 },
    "totalQuizzes": 2,
    "totalDurationSeconds": 13840
  },
  "assignments": {
    "total": 1,
    "submissions": {
      "total": 12,
      "byStatus": { "PENDING": 3, "GRADED": 8, "RETURNED": 1, "REJECTED": 0 }
    }
  },
  "enrollments": {
    "total": 57,
    "byStatus": { "ACTIVE": 40, "COMPLETED": 12, "DROPPED": 4, "SUSPENDED": 1 }
  },
  "progress": {
    "activeLearners": 52,
    "averageCompletionPercent": 63.46,
    "fullyCompletedLearners": 12,
    "completionRatePercent": 23.08
  },
  "ratings": {
    "average": 4.5,
    "count": 20,
    "distribution": { "1": 0, "2": 1, "3": 2, "4": 4, "5": 13 }
  }
}
```


Errors: `403` if the caller doesn't own the course; `404` if not found / soft-deleted.

`PATCH /courses/:courseId`

Update course. **Owner.** All fields optional; same shape as create plus `thumbnail?`.

Request body (example):


```
{ "title": "Advanced Kubernetes 2026", "price": 12900, "thumbnail": "courses/…/thumb.png" }
```


Response `200`: `{ "course_update": { "id": "…" } }`

`PATCH /courses/:courseId/status`

Publish / unpublish / archive. **Owner.**

Request body: `{ "status": "PUBLISHED" }` — `status` ∈ `DRAFT | PUBLISHED | ARCHIVED`.

Response `200`: `{ "course_update": { "id": "…" } }` **Errors:** `400` — publishing requires ≥1 section, each with ≥1 lecture.

`DELETE /courses/:courseId`

Soft delete. **Owner.** **Response** `200` : `{ "course_update": { "id": "…" } }`

Co-instructors

Method	Path	Body	Response `200/201`
`POST`	`/courses/:courseId/instructors`	`{ "userId": "uid-…" }`	`{ "courseInstructor_insert": { … } }`
`DELETE`	`/courses/:courseId/instructors/:userId`	—	`{ "courseInstructor_delete": { … } }`
`GET`	`/courses/:courseId/instructors`	—	`[ { "userId": "…", "name": "…" } ]`

All **Owner**.

---

6. Sections — `courses/:courseId/sections`

All **Instructor/Admin, owner**.

`POST /courses/:courseId/sections`

Body: `{ "title": "Module 1" }` → `201` `{ "courseSection_insert": { "id": "…" } }`

`PATCH /courses/:courseId/sections/:sectionId`

Body: `{ "title": "Module 1 — revised" }` → `200` `{ "courseSection_update": { "id": "…" } }`

`DELETE /courses/:courseId/sections/:sectionId`

→ `200` `{ "courseSection_delete": { "id": "…" } }`

`PUT /courses/:courseId/sections/reorder`

Body: `{ "orderedIds": ["<dashed-uuid-1>", "<dashed-uuid-2>"] }` (must be the complete set of the course's section IDs, dashed v4). → `200` `{ "success": true, "order": ["…","…"] }` **Errors:** `400` if `orderedIds` isn't exactly the existing set.

---

7. Lectures

Authoring routes are **Instructor/Admin, owner**. The read route is **User** (access-gated).

`POST /courses/:courseId/sections/:sectionId/lectures`

Request body:

```
{
  "title": "Pods & Deployments",
```

```
  "type": "VIDEO",
  "videoUrl": "courses/…/lectures/…/video.mp4",
  "content": "Markdown for READING lectures",
  "duration": 600,
  "transcriptUrl": "courses/…/transcript.pdf",
  "isFreePreview": false
}
```

Field	Type	Rules
`title`	string	required
`type`	enum	`VIDEO | READING | INTERACTIVE | QUIZ`
`videoUrl` / `content` / `transcriptUrl`	string	optional
`duration`	int	≥0 seconds, optional
`isFreePreview`	bool	optional

Response `201` : `{ "courseLecture_insert": { "id": "…" } }`

`PATCH /courses/:courseId/sections/:sectionId/lectures/:lectureId`

Same fields, all optional. → `200` `{ "courseLecture_update": { "id": "…" } }`

`DELETE /courses/:courseId/sections/:sectionId/lectures/:lectureId`

Soft delete. → `200` `{ "courseLecture_update": { "id": "…" } }`

`PUT /courses/:courseId/sections/:sectionId/lectures/reorder`

Body: `{ "orderedIds": ["…","…"] }` → `200` `{ "success": true, "order": [...] }`

`PATCH /courses/:courseId/lectures/:lectureId/move`

Move a lecture to another section. **Body:** `{ "targetSectionId": "<uuid>", "targetOrder": 2 }` (`targetOrder` ≥1) → `200` `{ "success": true }`

`GET /lectures/:lectureId`

Student-facing lecture content. **User**, access-gated (owner/admin or active/completed enrollee).

Response `200` :

```
{
  "id": "…",
  "title": "Pods & Deployments",
  "type": "VIDEO",
  "videoUrl": "https://signed-read-url…",
  "content": "…",
  "duration": 600,
  "transcriptUrl": "https://signed-read-url…",
  "isFreePreview": false
}
```

Errors: `403` if not enrolled and not a free preview · `404` if not found.

---

8. Enrollment

All **User**. Enroll is idempotent (reactivates a previously dropped enrollment).

`POST /courses/:courseId/enroll`

Response `201` (varies):

```
{ "id": "enrollment-uuid", "status": "ACTIVE", "alreadyEnrolled": true }
```

or `{ "id": "…", "status": "ACTIVE", "reactivated": true }`, or a fresh `{ "enrollment_insert": { "id": "…" } }` on first enroll.

`GET /courses/:courseId/enrollment`

My enrollment — **source of truth for "is the course completed"**. **Response** `200`: `{ "id": "…", "status": "ACTIVE" | "COMPLETED" | "DROPPED", "completedAt": "2026-07-01T…" | null }` or `null`.

`POST /missions/:missionId/enroll` · `GET /missions/:missionId/enrollment`

Same shapes as the course variants above, for missions.

---

9. Progress

All **User**.

`POST /lectures/:lectureId/progress` — playback heartbeat

High-frequency; queued, not written synchronously. Post every few seconds during playback. Completion is derived server-side at 90% — **never** send `completed` / `completedAt` here.

Request body:

```
{ "lastWatchedSecond": 145, "watchedSeconds": 130 }
```

Field	Type	Rules
`lastWatchedSecond`	int	≥0, required (furthest position)
`watchedSeconds`	int	≥0, optional (analytics)

Response `202`: `{ "accepted": true }`

`PUT /lectures/:lectureId/progress` — sync write

Synchronous progress write for when the client needs the result immediately. As with the queued POST above, **completion is derived server-side** (≥90% of the lecture's duration) — the legacy `completed` field is still accepted for backward compatibility but is **ignored**, so a client cannot assert completion. Lectures with no stored duration never auto-complete. **Body:** `{ "lastWatchedSecond": 600 }` → `200` `{ "success": true, "completed": false }` (`completed` in the response is the server-derived result for this write.)

`GET /lectures/:lectureId/progress`

Response `200`: `{ "lastWatchedSecond": 145, "completedAt": "2026-07-01T…" | null }` (defaults to `{ lastWatchedSecond: 0, completedAt: null }`).

`GET /courses/:courseId/progress`

Computed percentages. **Response** `200`:

```
{
  "totalLectures": 20,
  "completedLectures": 12,
  "overallPercentage": 60,
  "sections": [
    { "sectionId": "…", "sectionTitle": "Getting Started", "totalLectures": 5, "completedLectures": 5, "percentage": 100 }
  ]
}
```

---

10. Ratings & reviews — `courses/:courseId/ratings`

List and summary are **Public**; the `me` routes require **User** (upsert also requires **enrollment**).

`GET /courses/:courseId/ratings`

Paginated reviews. **Public. Query:** `?limit=20&offset=0` (`limit` 1–50, default 20; `offset` ≥0). **Response** `200`:

```
[
  { "rating": 5, "review": "Excellent!", "createdAt": "…", "updatedAt": "…", "user": { "name": "Jane", "avatar": "…" } }
]
```

`GET /courses/:courseId/ratings/summary`

Public. Response `200`:

```
{ "average": 4.6, "count": 128, "distribution": { "1": 2, "2": 3, "3": 10, "4": 40, "5": 73 } }
```

`GET /courses/:courseId/ratings/me`

My rating or `null`. **User. Response** `200`: `{ "rating": 5, "review": "…", "createdAt": "…", "updatedAt": "…" }` or `null`.

`PUT /courses/:courseId/ratings/me`

Create-or-update my rating. **User, enrolled. Body:** `{ "rating": 4, "review": "Solid course" }` — `rating` int 1–5; `review` optional ≤2000. **Response** `200` : `{ "courseRating_upsert": { … } }` **Errors:** `403` if not enrolled · `400` if course not published · `404` if course missing.

`DELETE /courses/:courseId/ratings/me`

User. → `200` `{ "courseRating_delete": { … } }`

---

11. Quizzes

11.1 Builder — Instructor/Admin, owner

`ResourceOwnershipGuard` resolves each leaf ID up to its owning course; the route's `:courseId` is not trusted.

`POST /courses/:courseId/lectures/:lectureId/quiz`

Body:

```
{ "passingScore": 70, "maxAttempts": 3, "timeLimitMinutes": 30 }
```

Field	Type	Rules
`passingScore`	number	0–100
`maxAttempts`	int	≥1
`timeLimitMinutes`	int	≥1, optional (untimed if omitted)

Response `201` : `{ "courseQuiz_insert": { "id": "…" } }`

`GET /courses/:courseId/quizzes/:quizId`

Answer-key view (**includes** `isCorrect`). **Builders only. Response** `200` :

```
{
  "id": "…", "passingScore": 70, "maxAttempts": 3, "timeLimitMinutes": 30,
  "adminQuestions": [
    { "id": "…", "questionText": "…", "points": 1, "explanation": "…", "order": 1,
      "options": [ { "id": "…", "text": "…", "isCorrect": true, "order": 1 } ] }
  ]
}
```

`PATCH /courses/:courseId/quizzes/:quizId`

Body: any of `{ passingScore?, maxAttempts?, timeLimitMinutes? }` → `200` `{ "courseQuiz_update": { "id": "…" } }`

`DELETE /courses/:courseId/quizzes/:quizId`

→ `200` `{ "courseQuiz_delete": { "id": "…" } }`

`POST /courses/:courseId/quizzes/:quizId/questions`

Body:

```
{
  "questionText": "Which is a controller?",
  "points": 2,
  "explanation": "Deployments manage ReplicaSets",
  "options": [
    { "text": "Deployment", "isCorrect": true },
    { "text": "Pod", "isCorrect": false }
  ]
}
```

`options` requires ≥2; `points` optional (≥0); `explanation` optional. **Response** `201`: `{ "quizQuestion_insert": { "id": "…" } }`

`PATCH /courses/:courseId/quizzes/:quizId/questions/:questionId`

Full desired option set diffs against existing (insert without `id`, update with `id`, delete anything omitted). **Body:**

```
{
  "questionText": "Updated?",
  "options": [
    { "id": "existing-opt-uuid", "text": "Deployment", "isCorrect": true },
    { "text": "New option", "isCorrect": false }
  ]
}
```

Response `200`: `{ "success": true }`

`DELETE /courses/:courseId/quizzes/:quizId/questions/:questionId`

Soft delete + re-normalizes remaining order. → `200` `{ "success": true }`

`PUT /courses/:courseId/quizzes/:quizId/questions/reorder`

Body: `{ "orderedIds": ["…","…"] }` → `200` `{ "success": true, "order": [...] }`

`PATCH …/questions/:questionId/options/:optionId`

Body: `{ "text": "…", "isCorrect": true }` (both optional) → `200` `{ "quizOption_update": { … } }`

`DELETE …/questions/:questionId/options/:optionId`

→ `200` `{ "quizOption_delete": { … } }`

`PUT …/questions/:questionId/options/reorder`

Body: `{ "orderedIds": ["…","…"] }` → `200` `{ "success": true, "order": [...] }`

11.2 Taking — User (enrolled); attempt-owner enforced

`GET /courses/:courseId/lectures/:lectureId/quiz`

Resolves the quiz attached to a QUIZ-type lecture — the entry point when the client holds only a `lectureId` (e.g. rendering a lecture) and needs to discover the quiz and its `id`. Student-safe view (**no** `isCorrect`), identical shape to the `…/quizzes/:quizId/questions` response below. Access is gated against the lecture's real owning course (owner/admin previewing, or active enrollee); the route's `:courseId` is not trusted. **Response** `200`:

```
{
  "id": "…", "passingScore": 70, "maxAttempts": 3, "timeLimitMinutes": 30,
  "questions": [
    { "id": "…", "questionText": "…", "points": 2,
      "options": [ { "id": "…", "text": "Deployment" } ] }
  ]
}
```

Errors: `404` lecture not found / lecture has no quiz · `403` not owner/admin and no active enrollment.

`GET /courses/:courseId/quizzes/:quizId/questions`

Student-safe view (**no** `isCorrect`). Render the quiz from this. **Response** `200`:

```
{
  "id": "…", "timeLimitMinutes": 30,
  "questions": [
    { "id": "…", "questionText": "…", "points": 2, "order": 1,
      "options": [ { "id": "…", "text": "Deployment", "order": 1 } ] }
  ]
}
```

`POST /courses/:courseId/quizzes/:quizId/attempts`

Start an attempt. **Response** `201`: `{ "quizAttempt_insert": { "id": "attempt-uuid" } }` **Errors:** `403` no active/completed enrollment / max attempts exceeded · `400` an attempt is already in progress.

`POST …/attempts/:attemptId/answers`

Submit one answer. **Body:** `{ "questionId": "<uuid>", "selectedOptionId": "<uuid>" }` **Response** `201`: `{ "quizAnswer_upsert": { … } }` **Errors:** `403` not your attempt · `400` attempt already completed · `410` **Gone** — session timed out (auto-graded); response includes the grade payload.

`POST …/attempts/:attemptId/finish`

Finalize + grade. **Response** `201`:

```
{
  "score": 85.71,
  "passed": true,
  "totalPoints": 7,
  "earnedPoints": 6,
  "correctCount": 5,
  "answeredCount": 6,
  "totalQuestions": 7
}
```

If a concurrent finish/timeout already graded it, the same shape plus `"alreadyFinalized": true`.

`POST …/attempts/:attemptId/heartbeat`

Keep a timed attempt alive / check remaining time. **Response** `201`:

- Untimed: `{ "active": true, "remainingSeconds": null }`
- Timed, running: `{ "active": true, "remainingSeconds": 842 }`
- Already done: `{ "active": false, "message": "Attempt already completed" }`
- Timed out (auto-graded): `{ "active": false, "message": "Session timed out", "score": …, "passed": …, … }`

---

12. Missions (authoring) — `missions`

All **Instructor/Admin**; mutating a specific mission requires **owner**.

`POST /missions`

Body:

```
{
  "title": "Incident Response Drill",
  "scenarioDescription": "A production outage…",
  "durationMinutes": 90,
  "categoryId": "<uuid>",
  "difficulty": "ADVANCED",
  "skillsTested": ["debugging", "communication"],
  "courseId": "<uuid>"
}
```

Field	Type	Rules
`title` / `scenarioDescription`	string	required
`durationMinutes`	int	≥1
`categoryId`	UUID	required
`difficulty`	enum	`BEGINNER` | `INTERMEDIATE` | `ADVANCED` | `EXPERT`
`skillsTested`	string[]	required
`courseId`	UUID	optional (link to a course; ownership checked)

Response `201`: `{ "mission_insert": { "id": "…" } }`

`GET /missions`

Missions I authored. **Response** `200`: array.

`GET /missions/:missionId`

Owner. Full mission object.

`PATCH /missions/:missionId`

Owner. Same fields, all optional. → `200` `{ "mission_update": { "id": "…" } }`

`DELETE /missions/:missionId`

Owner. Soft delete. → `200` `{ "mission_update": { "id": "…" } }`

---

13. Micro-lessons — `missions/:missionId/micro-lessons`

All **Instructor/Admin, owner**.

`POST /missions/:missionId/micro-lessons`

Body:

```
{ "title": "Brief", "type": "READING", "assetUrl": "…", "content": "…", "durationSeconds": 120 }
```

`type` ∈ `VIDEO | READING | INTERACTIVE | QUIZ`; `assetUrl` / `content` / `durationSeconds` optional (`durationSeconds` ≥0). **Response** `201`: `{ "missionMicroLesson_insert": { "id": "…" } }`

`PATCH /missions/:missionId/micro-lessons/:microLessonId`

Same fields, optional. → `200` `{ "missionMicroLesson_update": { "id": "…" } }`

`DELETE /missions/:missionId/micro-lessons/:microLessonId`

→ `200` `{ "missionMicroLesson_delete": { "id": "…" } }`

`PUT /missions/:missionId/micro-lessons/reorder`

Body: `{ "orderedIds": ["…","…"] }` → `200` `{ "success": true, "order": [...] }`

---

14. Mission templates — `missions/:missionId/templates`

`POST /missions/:missionId/templates`

Instructor/Admin, owner. `multipart/form-data` (not JSON).

Form fields:

Field	Type	Rules
`file`	file	required; `.pdf` / `.docx` / `.xlsx`, ≤50 MB. Validated by `FileValidationPipe` (MIME + magic-bytes + size) and hard-capped at the Multer layer
`title`	string	1–200 chars, required

Response `201` : `{ "missionTemplate_insert": { "id": "…", "fileUrl": "…" } }` **Errors:** `413 Payload Too Large` if the file exceeds 50 MB; `400 Bad Request` for a disallowed type/extension or contents that don't match the declared type.

`GET /missions/:missionId/templates`

User, access-gated: the mission's owner/admin **or** a caller with an `ACTIVE` or `COMPLETED` enrollment in the mission. `fileUrl` is a short-lived signed read URL. **Response** `200` : array of `{ id, title, fileUrl, fileType }` . **Errors:** `403` if the caller neither owns the mission nor has an active/completed enrollment.

`DELETE /missions/:missionId/templates/:templateId`

Instructor/Admin, owner. → `200` `{ "missionTemplate_delete": { … } }`

---

15. Mission rubric — `missions/:missionId/rubric`

All **Instructor/Admin, owner**.

`POST /missions/:missionId/rubric`

Body: `{ "criterionName": "Clarity", "description": "…", "maxPoints": 10 }` (`description` optional; `maxPoints` ≥0). **Response** `201` : `{ "rubricCriterion_insert": { "id": "…" } }`

`PATCH /missions/:missionId/rubric/:criterionId`

Same fields, optional. → `200` `{ "rubricCriterion_update": { "id": "…" } }`

`DELETE /missions/:missionId/rubric/:criterionId`

→ `200` `{ "rubricCriterion_delete": { "id": "…" } }`

`PUT /missions/:missionId/rubric/reorder`

Body: `{ "orderedIds": ["…","…"] }` → `200` `{ "success": true, "order": [...] }`

---

16. Mission submissions & grading

`POST /missions/:missionId/submissions`

User (student), enrolled. Upload the file first (§18), then submit its URL. **Body:** `{ "fileUrl": "https://…/deliverable.pdf" }` (required, ≤2000 chars). **Response** `201` : `{ "missionSubmission_insert": { "id": "…" } }` **Errors:** `403` if not actively or completely enrolled in the mission.

`GET /me/submissions`

User. My submissions + scores + feedback. **Response** `200` :

```
[
  { "id": "…", "missionId": "…", "fileUrl": "…", "status": "GRADED", "feedback": "…",
```

```
    "scores": [ { "criterionId": "…", "score": 8 } ], "submittedAt": "…" }
]
```

`GET /submissions/pending`

Instructor/Admin. Submissions awaiting grading. **Response** `200`: array of submission objects.

`POST /submissions/:submissionId/grade`

Instructor/Admin. Body:

```
{
  "feedback": "Good structure, tighten the summary.",
  "scores": [ { "criterionId": "<uuid>", "score": 8 }, { "criterionId": "<uuid>", "score": 6 } ]
}
```

Both optional; each `score` ≥0. Grading marks the student's mission enrollment `COMPLETED`. **Response** `201`: `{ "success": true }`

---

17. File uploads — `uploads`

Instructor/Admin. Signed-URL, direct-to-storage pattern (see §18 for the full flow).

`POST /uploads/signed-url`

Get a signed PUT URL. The server derives the storage path — the client never supplies one. **Body:**

```
{
  "entityType": "LECTURE_VIDEO",
  "courseId": "<uuid>",
  "lectureId": "<uuid>",
  "filename": "lesson1.mp4",
  "contentType": "video/mp4"
}
```

Field	Type	Rules
`entityType`	enum	`COURSE_THUMBNAIL | LECTURE_VIDEO | LECTURE_TRANSCRIPT`
`courseId`	UUID	required (authorizes caller)
`lectureId`	UUID	required for `LECTURE_*` types
`filename`	string	`^[\w.\- ]{1,200}$`
`contentType`	string	must match the entity's allowed MIME list

Response `201`:

```
{ "uploadUrl": "https://storage.googleapis.com/…?X-Goog-Signature=…", "downloadUrl": "https://…", "storagePath": "courses/.
```

Errors: `403` not course owner · `400` missing `lectureId` / disallowed content type.

`POST /uploads/confirm`

Confirm after the PUT; persists the reference. For `LECTURE_VIDEO` this also auto-enqueues transcoding. **Body:**

```
{ "entityType": "LECTURE_VIDEO", "courseId": "<uuid>", "lectureId": "<uuid>", "storagePath": "courses/…/video/uuid-lesson1.
```

Response `201`:

```
{ "downloadUrl": "https://signed-read-url…", "storagePath": "courses/…/uuid-lesson1.mp4", "transcode": { "jobId": "transcoc
```

(`transcode` present only for `LECTURE_VIDEO`.) **Errors:** `400` path outside course prefix / file not found / lecture not in course.

---

18. File upload flow (not a normal JSON POST)

1. `POST /uploads/signed-url` → `{ uploadUrl, downloadUrl, storagePath }`.
2. **`PUT` the raw bytes directly to `uploadUrl`** (Google Storage) with header `Content-Type: <same contentType>`. This does **not** go through the API and needs **no** Firebase token.
3. `POST /uploads/confirm` with the returned `storagePath` to persist the reference.
4. For lecture videos, confirm auto-queues transcoding (or call `POST …/video/transcode` explicitly).

Exception: mission templates (`POST /missions/:missionId/templates`) are a direct `multipart/form-data` upload through the API.

---

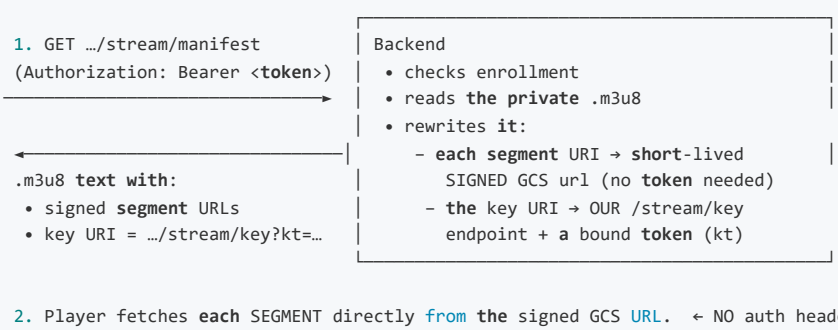
19. Video streaming (playback)

> **Read §19.1 before writing any player code.** Secure video is the one part of this API you can't treat as a plain URL. Getting the auth wiring wrong is the #1 cause of "the video won't play."

19.1 How the streaming system works (the mental model)

Videos are **not** served as a plain downloadable file. They are packaged into **AES-128-encrypted HLS** (an `.m3u8` master playlist + many small `.ts` segments + one encryption key), and all of it lives in a **private** Google Cloud Storage bucket that has **no public URLs**. This defeats direct download, hotlinking, and link-sharing. (It does **not** stop screen recording — that's what the watermark below is for.)

Because nothing is public, playback is a short, guarded handshake between the player and three API endpoints:



```
3. Player fetches the KEY:  GET …/stream/key?kt=…                ← MUST send Authorization: Bearer <token>
   Backend re-checks the Firebase token + live enrollment + the kt token, returns the raw 16 bytes,
   the player decrypts the segments, and plays.
```

The one rule that matters for the frontend: the manifest **and** the key requests hit **our API** and are both protected by Firebase auth — so the player must attach `Authorization: Bearer <idToken>` to them. The segment requests go to `storage.googleapis.com` with a pre-signed URL and must **not** carry the Authorization header. §19.5 shows exactly how to configure this.

Lifetimes (defaults): signed segment URLs and the `kt` key token both live **6 hours** (`STREAM_URL_TTL_MINUTES=360`). The Firebase ID token lives **1 hour** — so for long sessions the player must use a *fresh* Firebase token on the key request (Firebase SDKs auto-refresh; just call `getIdToken()` at request time, §19.5).

19.2 `GET /lectures/:lectureId/stream/manifest?v=<variant>`

User — access-gated: allowed if the lecture is a free preview, **or** the caller owns/admins the course, **or** the caller has an `ACTIVE` or `COMPLETED` enrollment.

- **Query:** `v` (optional) selects a sub-playlist/variant for adaptive-bitrate masters. **You never build this yourself** — the master playlist the API returns already contains the correct `?v=…` URLs; the player follows them automatically.
- **Response** `200` — headers `Content-Type: application/vnd.apple.mpegurl`, `Cache-Control: no-store`. Body is the rewritten `.m3u8` text. **Give this URL to an HLS-capable player; do not parse it yourself.**
- **Errors:** `403` not enrolled · `404` lecture/video not found · `400` the lecture's video isn't packaged for secure streaming (legacy absolute URL).

Example (illustrative — a media playlist after rewriting):

```
#EXTM3U
#EXT-X-VERSION:3
#EXT-X-TARGETDURATION:6
#EXT-X-KEY:METHOD=AES-128,URI="https://api.optimatech.io/api/v1/lectures/<id>/stream/key?kt=eyJ…"
#EXTINF:6.0,
https://storage.googleapis.com/<bucket>/…/seg0.ts?X-Goog-Signature=…
#EXTINF:6.0,
https://storage.googleapis.com/<bucket>/…/seg1.ts?X-Goog-Signature=…
#EXT-X-ENDLIST
```

19.3 `GET /lectures/:lectureId/stream/watermark`

User, same access gate as the manifest. Returns a signed, per-session forensic watermark the player should overlay on the video (to trace any leaked recording back to the user). Reposition it periodically so it can't be cropped out.

Response `200`:

```
{
  "text": "jane@example.com • 2026-07-04T10:15:00.000Z",
  "uid": "firebase-uid-123",
  "email": "jane@example.com",
  "lectureId": "…",
  "sessionId": "b1f2…-uuid",
  "issuedAt": "2026-07-04T10:15:00.000Z",
  "expiresAt": "2026-07-04T16:15:00.000Z",
  "signature": "hmac-base64url"
}
```

Frontend uses `text` only (display it as an overlay). The other fields are for server-side forensics; you don't need to validate the signature client-side.

19.4 `GET /lectures/:lectureId/stream/key?kt=<keyToken>`

User, enrolled. Returns the raw 16-byte AES-128 content key. **The player calls this automatically** while playing — you normally never call it by hand. It requires **both** a valid Firebase token (header) **and** the `kt` token (query, embedded in the manifest by the backend).

- **Response** `200` — `Content-Type: application/octet-stream`; body is the raw 16 key bytes.
- **Errors:** `400` missing `kt` · `403` `kt` doesn't match this user/lecture or enrollment was revoked · `401` Firebase token missing/expired.

19.5 Frontend implementation guide — playing a secure lecture video

Step 1 — decide the lecture is a video. From `GET /lectures/:lectureId` (§7) or the curriculum, `type === "VIDEO"`. Build the manifest URL: `"$baseUrl/lectures/$lectureId/stream/manifest"`.

Step 2 — feed the manifest URL to an HLS player, with a token interceptor that adds the Firebase header to API requests only. This is the whole trick. Segment requests must stay header-free.

Web (`hls.js`): use `xhrSetup` to attach the token only to requests that hit our API host (manifest + key), and never to `storage.googleapis.com` (segments):

```
import Hls from 'hls.js';

const API_HOST = new URL(apiBaseUrl).host; // e.g. api.optimatech.io
const manifestUrl = `${apiBaseUrl}/lectures/${lectureId}/stream/manifest`;

const hls = new Hls({
  xhrSetup: async (xhr, url) => {
    // Attach the Firebase token ONLY to our API (manifest + key), never to GCS segments.
    if (new URL(url).host === API_HOST) {
      const token = await firebase.auth().currentUser.getIdToken(); // auto-refreshes near expiry
      xhr.setRequestHeader('Authorization', `Bearer ${token}`);
    }
  },
});
hls.loadSource(manifestUrl);
hls.attachMedia(videoEl);
```

Safari (native HLS, no hls.js) can't add headers to key requests; on Safari, load through hls.js as above rather than setting `<video src>` directly.

Flutter (`better_player`, recommended for header support): `better_player` lets you pass headers that it forwards to the manifest and key requests. Fetch a fresh token right before creating the data source:

```
final token = await FirebaseService.currentUser!.getIdToken();
final dataSource = BetterPlayerDataSource(
  BetterPlayerDataSourceType.network,
  '$apiBaseUrl/lectures/$lectureId/stream/manifest',
  videoFormat: BetterPlayerVideoFormat.hls,
  headers: { 'Authorization': 'Bearer $token' },
);
```

> ⚠️ **Two things to watch on mobile:**
>
> 1. **Token expiry on long videos.** The header above is captured once. Because the Firebase token expires in ~1 hour but the key token lives 6 hours, a >1-hour continuous session can hit a `401` on a later key fetch. Mitigate by getting a fresh token when (re)creating the player, and re-initializing the data source if playback fails with an auth error. (For most lectures under an hour this never triggers.)

> 2. **Header leaking to GCS segments.** Native HLS players apply the same headers to every request, including the signed GCS segment URLs. A stray `Authorization` header on a pre-signed GCS URL is normally ignored, but if you see segment `400`s, that's the cause — prefer a player/config that scopes headers to the manifest+key host, or proxy segments. Test end-to-end early.

Step 3 — overlay the watermark. Call `GET …/stream/watermark`, and render `text` as a semi-transparent overlay on top of the video widget; move it every ~10–20s.

Step 4 — report progress while playing via `POST /lectures/:lectureId/progress` (§9), and mark complete when done.

Do not: cache/persist the manifest, segment, or key URLs — they expire and are per-session. Always re-request the manifest when the user opens the lecture.

19.6 `POST /courses/:courseId/lectures/:lectureId/stream/register`

Instructor/Admin, owner. Registers already-uploaded HLS assets: verifies the master playlist and AES key exist in storage, then points the lecture's `videoUrl` at the master. Usually the transcoding pipeline (§20) does this automatically; this is the manual fallback. **Response** `201`: `{ "success": true, "videoPath": "courses/…/lectures/…/hls/master.m3u8" }` **Errors:** `400` master playlist or content key not yet uploaded.

---

20. Video transcoding — Instructor/Admin, owner

`POST /courses/:courseId/lectures/:lectureId/video/transcode`

Queue secure-HLS packaging. **Body:** `{ "sourcePath": "courses/<c>/lectures/<l>/video/<file>.mp4" }` (≤1024 chars; verified to belong to this lecture). **Response** `202`: `{ "jobId": "transcode-<lectureId>", "status": "queued" }`

`GET /courses/:courseId/lectures/:lectureId/video/transcode/status`

Poll transcode status. **Response** `200`:

```
{ "status": "active", "progress": 42, "failedReason": null }
```

`status` is a BullMQ job state (`queued | active | completed | failed | …`). **Errors:** `404` no transcode job for this lecture.

---

21. AI Service Integration — `ai`

Endpoints that bridge the LMS Core to the external AI Service for video indexing and RAG chat.

`GET /ai/lectures/:lectureId/status`

Poll the current AI indexing state for a video lecture. **User. Response** `200`: `{ "status": "PENDING" | "PROCESSING" | "READY" | "READY_WITH_WARNINGS" | "FAILED" | "UNKNOWN" }`

`POST /ai/lectures/:lectureId/chat`

Proxy a student's message to the AI Service for RAG chat. **User. Body:** `{ "message": "What is a residual?", "playbackMs": 75000, "sessionId": "<uuid>" }` **Response** `200`: `{ "answer": "...", "grounded": true, "citations": [...] }`

`POST /ai/lectures/:lectureId/retry`

Retry AI ingestion for a failed video processing job. **Instructor/Admin, owner. Response** `201` : `{ "success": true, "aiJobId": "..." }`

---

22. Common flows

Student watches a course

1. `GET /catalog/courses` → pick a course
2. `GET /catalog/courses/:id/curriculum` → outline + rating summary
3. `POST /courses/:id/enroll`
4. `GET /lectures/:lectureId` → content; for video point the player at `GET /lectures/:lectureId/stream/manifest`
5. During playback: `POST /lectures/:lectureId/progress` every few seconds
6. `GET /courses/:id/progress` for the bar; `GET /courses/:id/enrollment` for completed state
7. After finishing: `PUT /courses/:id/ratings/me`

Instructor authors a course

1. `POST /courses` → 2. `POST /courses/:id/sections` → 3. add lectures → 4. (video) upload via §18 → transcode → 5. `POST …/stream/register` → 6. `PATCH /courses/:id/status {status:"PUBLISHED"}`

Student takes a quiz

1. `GET …/lectures/:lectureId/quiz` (resolve quiz from the lecture) → 2. `POST …/attempts` → 3. `POST …/attempts/:id/answers` (per answer) → 4. `POST …/attempts/:id/heartbeat` (if timed) → 5. `POST …/attempts/:id/finish`

> Clients that already hold the `quizId` may skip step 1 and use `GET …/quizzes/:quizId/questions` instead.

---

23. Client-called Cloud Functions (Firebase SDK — NOT REST)

> **Read this if you build sign-up or the admin role UI.** These live in a **separate** Firebase Cloud Functions deployment (`/functions`), **not** the NestJS backend. You do **not** call them with `fetch` /a base URL — you invoke them through the Firebase SDK's `httpsCallable` . They are **not** included in the 90 REST routes above.

Region: all functions are deployed to `europe-west3` — you must pass this region when getting the callable, or the call fails.

Invocation pattern (web):

```
import { getFunctions, httpsCallable } from 'firebase/functions';

const fns = getFunctions(app, 'europe-west3');          // region is required
const registerStudent = httpsCallable(fns, 'registerStudent');
const res = await registerStudent({ name, email, password, nationalId });
// res.data === { success: true, uid: "…" }
```

Invocation pattern (Flutter):

```
final fns = FirebaseFunctions.instanceFor(region: 'europe-west3');
final res = await fns.httpsCallable('registerStudent').call({
  'name': name, 'email': email, 'password': password, 'nationalId': nationalId,
```

```
});
// res.data == { 'success': true, 'uid': '…' }
```

Errors come back as a `FirebaseFunctionsException` with a `code` and `message` (not an HTTP status). Show `message` to the user. Relevant codes: `resource-exhausted` (rate-limited), `invalid-argument` (validation failed — `message` is the first failing rule), `already-exists` (duplicate email), `unauthenticated` (not signed in), `permission-denied` (not admin / forbidden action), `not-found`, `internal` (server error).

22.1 `registerStudent` — email/password sign-up

The **only** way to register a student with email + password. It creates the Firebase Auth account, the DB user row (with an encrypted national ID), assigns the default `student` role, and sets custom claims — all in one call. **No auth token required** (the user doesn't exist yet).

Request payload:

```
{ "name": "Jane Doe", "email": "jane@example.com", "password": "P@ssw0rd!", "nationalId": "12345678901234" }
```

Field	Type	Rules
`name`	string	2–50 chars, trimmed
`email`	string	valid email (lower-cased/trimmed server-side)
`password`	string	8–64 chars, **≥1 uppercase, ≥1 number, ≥1 special char**
`nationalId`	string	**exactly 14 digits** (encrypted at rest server-side; never returned)

Success `→ res.data`:

```
{ "success": true, "uid": "firebase-uid-123" }
```

Errors: `invalid-argument` (any rule above; `message` names the exact failure) · `already-exists` (email taken) · `resource-exhausted` (>10 attempts/min per IP) · `internal`.

After success: the user is created but **not signed in** — follow up with `signInWithEmailAndPassword` on the client, then you can call the REST API (start with `POST /me/sync`, §2).

> **Social / OAuth sign-in needs none of this.** When a user first signs in with Google (or any provider), the background `syncClaims` trigger auto-creates their DB row + default `student` claim. The frontend just does the normal Firebase social sign-in — no extra call.

22.2 `changeUserRole` — admin sets another user's role

Admin-only. Rewrites the target user's role rows in the DB **and** their token claims. **Requires the caller to be signed in as an admin** (the SDK attaches the token automatically).

Request payload:

```
{ "targetUid": "firebase-uid-of-target", "newRole": "instructor" }
```

Field	Type	Rules
`targetUid`	string	non-empty; the user to modify
`newRole`	enum	`student | instructor | admin`

Success `→ res.data` :

```
{ "success": true, "message": "User successfully updated to instructor." }
```

Errors: `unauthenticated` (not signed in) · `permission-denied` (caller isn't admin, **or** an admin tried to change their own role — not allowed) · `invalid-argument` (bad role/uid) · `not-found` (target user doesn't exist) · `resource-exhausted` (>20 attempts/min per admin) · `internal` .

> **Two ways to change roles — confirm with backend which to use.** This callable and the REST `PUT /admin/users/:uid/role` (§3) both exist. The callable updates **both** the DB role rows and the token claims; the REST route sets the claim only. For the admin UI, prefer whichever the backend team designates as canonical.

22.3 Reading the current user's role on the client

After sign-in, roles live in the ID token claims — read them to gate UI (don't call an endpoint for this):

```
const { claims } = await firebase.auth().currentUser.getIdTokenResult();
const roles = claims.roles ?? [];          // e.g. ["instructor"] — NOTE: array
const isAdmin = roles.includes('admin');
```

The `roles` **array is authoritative** — the backend and Data Connect read only it. Legacy single-role claims (`claims.role` , `claims.admin`) are no longer honored anywhere; don't read them. After a role change, the token must refresh before new claims appear: `await currentUser.getIdToken(true)` (force refresh) or re-sign-in.

22.4 Background triggers (no frontend work)

For completeness — these run automatically inside Firebase and the frontend never calls them:

- `syncClaims` — on user creation: provisions the DB row + default `student` claim (covers OAuth sign-ups).
- `cleanupUserData` — on user deletion: wipes the user's DB row and `users/<uid>/` storage.

Appendix A — Complete endpoint index (all 95 routes)

Every route in the backend, prefixed with `/api/v1` . Use it as a build checklist. Auth: **P** = Public · **U** = User · **I/A** = Instructor/Admin · **Own** = must own the resource · **Adm** = Admin only.

#	Method	Path	Auth	§
1	GET	`/health`	P	1
2	GET	`/health/ready`	P	1
3	POST	`/me/sync`	U	2
4	GET	`/me`	U	2

#	Method	Path	Auth	§
5	PATCH	`/me/profile`	U	2
6	PUT	`/admin/users/:uid/role`	Adm	3
7	GET	`/catalog/courses`	P	4
8	GET	`/catalog/courses/:courseId/curriculum`	U	4
9	GET	`/courses/search`	U	5
10	GET	`/courses/categories`	U	5
11	GET	`/courses/popular`	U	5
12	GET	`/courses/recommended`	U	5
13	GET	`/courses/continue-learning`	U	5
14	GET	`/courses/:courseId/videos`	U	5
15	POST	`/courses`	I/A	5
16	GET	`/courses`	I/A	5
17	GET	`/courses/:courseId`	I/A Own	5
18	GET	`/courses/:courseId/overview`	I/A Own	5
19	PATCH	`/courses/:courseId`	I/A Own	5
20	PATCH	`/courses/:courseId/status`	I/A Own	5
21	DELETE	`/courses/:courseId`	I/A Own	5
22	POST	`/courses/:courseId/instructors`	I/A Own	5
23	DELETE	`/courses/:courseId/instructors/:userId`	I/A Own	5
24	GET	`/courses/:courseId/instructors`	I/A Own	5
25	POST	`/courses/:courseId/sections`	I/A Own	6
26	PATCH	`/courses/:courseId/sections/:sectionId`	I/A Own	6
27	DELETE	`/courses/:courseId/sections/:sectionId`	I/A Own	6
28	PUT	`/courses/:courseId/sections/reorder`	I/A Own	6
29	POST	`/courses/:courseId/sections/:sectionId/lectures`	I/A Own	7
30	PATCH	`/courses/:courseId/sections/:sectionId/lectures/:lectureId`	I/A Own	7
31	DELETE	`/courses/:courseId/sections/:sectionId/lectures/:lectureId`	I/A Own	7
32	PUT	`/courses/:courseId/sections/:sectionId/lectures/reorder`	I/A Own	7
33	PATCH	`/courses/:courseId/lectures/:lectureId/move`	I/A Own	7
34	GET	`/lectures/:lectureId`	U	7
35	POST	`/courses/:courseId/enroll`	U	8
36	GET	`/courses/:courseId/enrollment`	U	8

#	Method	Path	Auth	§
37	POST	`/missions/:missionId/enroll`	U	8
38	GET	`/missions/:missionId/enrollment`	U	8
39	POST	`/lectures/:lectureId/progress`	U	9
40	PUT	`/lectures/:lectureId/progress`	U	9
41	GET	`/lectures/:lectureId/progress`	U	9
42	GET	`/courses/:courseId/progress`	U	9
43	GET	`/courses/:courseId/ratings`	P	10
44	GET	`/courses/:courseId/ratings/summary`	P	10
45	GET	`/courses/:courseId/ratings/me`	U	10
46	PUT	`/courses/:courseId/ratings/me`	U enrolled	10
47	DELETE	`/courses/:courseId/ratings/me`	U	10
48	POST	`/courses/:courseId/lectures/:lectureId/quiz`	I/A Own	11.1
49	GET	`/courses/:courseId/quizzes/:quizId`	I/A Own	11.1
50	PATCH	`/courses/:courseId/quizzes/:quizId`	I/A Own	11.1
51	DELETE	`/courses/:courseId/quizzes/:quizId`	I/A Own	11.1
52	POST	`/courses/:courseId/quizzes/:quizId/questions`	I/A Own	11.1
53	PATCH	`/courses/:courseId/quizzes/:quizId/questions/:questionId`	I/A Own	11.1
54	DELETE	`/courses/:courseId/quizzes/:quizId/questions/:questionId`	I/A Own	11.1
55	PUT	`/courses/:courseId/quizzes/:quizId/questions/reorder`	I/A Own	11.1
56	PATCH	`/courses/:courseId/quizzes/:quizId/questions/:questionId/options/:optionId`	I/A Own	11.1
57	DELETE	`/courses/:courseId/quizzes/:quizId/questions/:questionId/options/:optionId`	I/A Own	11.1
58	PUT	`/courses/:courseId/quizzes/:quizId/questions/:questionId/options/reorder`	I/A Own	11.1
59	GET	`/courses/:courseId/lectures/:lectureId/quiz`	U enrolled	11.2
60	GET	`/courses/:courseId/quizzes/:quizId/questions`	U enrolled	11.2
61	POST	`/courses/:courseId/quizzes/:quizId/attempts`	U enrolled	11.2
62	POST	`/courses/:courseId/quizzes/:quizId/attempts/:attemptId/answers`	U (owner)	11.2
63	POST	`/courses/:courseId/quizzes/:quizId/attempts/:attemptId/finish`	U (owner)	11.2
64	POST	`/courses/:courseId/quizzes/:quizId/attempts/:attemptId/heartbeat`	U (owner)	11.2

#	Method	Path	Auth	§
65	POST	`/missions`	I/A	12
66	GET	`/missions`	I/A	12
67	GET	`/missions/:missionId`	I/A Own	12
68	PATCH	`/missions/:missionId`	I/A Own	12
69	DELETE	`/missions/:missionId`	I/A Own	12
70	POST	`/missions/:missionId/micro-lessons`	I/A Own	13
71	PATCH	`/missions/:missionId/micro-lessons/:microLessonId`	I/A Own	13
72	DELETE	`/missions/:missionId/micro-lessons/:microLessonId`	I/A Own	13
73	PUT	`/missions/:missionId/micro-lessons/reorder`	I/A Own	13
74	POST	`/missions/:missionId/templates`	I/A Own	14
75	GET	`/missions/:missionId/templates`	U enrolled	14
76	DELETE	`/missions/:missionId/templates/:templateId`	I/A Own	14
77	POST	`/missions/:missionId/rubric`	I/A Own	15
78	PATCH	`/missions/:missionId/rubric/:criterionId`	I/A Own	15
79	DELETE	`/missions/:missionId/rubric/:criterionId`	I/A Own	15
80	PUT	`/missions/:missionId/rubric/reorder`	I/A Own	15
81	POST	`/missions/:missionId/submissions`	U enrolled	16
82	GET	`/me/submissions`	U	16
83	GET	`/submissions/pending`	I/A	16
84	POST	`/submissions/:submissionId/grade`	I/A	16
85	POST	`/uploads/signed-url`	I/A Own	17
86	POST	`/uploads/confirm`	I/A Own	17
87	GET	`/lectures/:lectureId/stream/manifest`	U enrolled	19.2
88	GET	`/lectures/:lectureId/stream/watermark`	U enrolled	19.3
89	GET	`/lectures/:lectureId/stream/key`	U enrolled	19.4
90	POST	`/courses/:courseId/lectures/:lectureId/stream/register`	I/A Own	19.6

#	Method	Path	Auth	§
91	POST	`/courses/:courseId/lectures/:lectureId/video/transcode`	I/A Own	20
92	GET	`/courses/:courseId/lectures/:lectureId/video/transcode/status`	I/A Own	20
93	GET	`/ai/lectures/:lectureId/status`	U	21
94	POST	`/ai/lectures/:lectureId/chat`	U	21
95	POST	`/ai/lectures/:lectureId/retry`	I/A Own	21

> **Route total: 95** distinct handlers across 20 controller files. This index was generated by scanning every `@Get/@Post/@Put/@Patch/@Delete` decorator in `src/**/*.controller.ts` — it is the full set. The **Source** column links each route to its handler in the codebase (`file:line`; line numbers for the health routes are as of the 2026-07-28 scan, the rest as of 2026-07-05, and may drift). Breakdown: health 2, me 3, admin 1, catalog 2, courses 16, sections 4, lectures 6, enrollment 4, progress 4, ratings 5, quizzes 17, missions 5, micro-lessons 4, templates 3, rubric 4, submissions 4, uploads 2, streaming 4, transcoding 2, ai 3.

Appendix B — Controller & service source map

Where each route group lives in the backend, and the service it delegates to. Paths are relative to `backend/`. Controllers only validate + authorize (guards/DTOs); the **service** holds the business logic and the Data Connect / Firebase calls. DTOs (request-body shapes + validation rules) live in each module's `dto/` folder next to its controller.

Controller	Base path	Routes	Delegates to
health.controller.ts	`/health`	2	`HealthService` (+ `ShutdownService`)
users.controller.ts	`/me`	3	`UsersService`
admin.controller.ts	`/admin/users`	1	`FirebaseService`
catalog.controller.ts	`/catalog`	2	`CatalogService`
courses.controller.ts	`/courses`	16	`CoursesService`
sections.controller.ts	`/courses/:courseId/sections`	4	`SectionsService`
lectures.controller.ts	`/` (multiple)	6	`LecturesService`
enrollment.controller.ts	`/` (multiple)	4	`EnrollmentService`
progress.controller.ts	`/` (multiple)	4	`ProgressService` (+ BullMQ lecture-progress queue)
ratings.controller.ts	`/courses/:courseId/ratings`	5	`RatingsService`
quizzes.controller.ts	`/` (multiple)	16	`QuizzesService`
missions.controller.ts	`/missions`	5	`MissionsService`
micro-lessons.controller.ts	`/missions/:missionId/micro-lessons`	4	`MicroLessonsService`
templates.controller.ts	`/missions/:missionId/templates`	3	`TemplatesService`
rubric.controller.ts	`/missions/:missionId/rubric`	4	`RubricService`
submissions.controller.ts	`/` (multiple)	4	`SubmissionsService`

Controller	Base path	Routes	Delegates to
uploads.controller.ts	`/uploads`	2	`UploadsService`, `DataConnectService`
streaming.controller.ts	`/` (multiple)	4	`StreamingService`
transcoding.controller.ts	`/` (multiple)	2	`TranscodingService`
ai.controller.ts	`/ai`	3	`AiService`

Totals: 95 routes across 20 controllers. Guards live in `src/common/guards/` (`FirebaseAuthGuard` → `RolesGuard` → `CourseAccessGuard` / `ResourceOwnershipGuard`); shared decorators in `src/common/decorators/`.